

University of Pittsburgh

Introduction to Operating Systems
CS/COE 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Project 1: due on 2/18
  - Homework 5: due 2/21
  - Lab 2: due on 2/28

Previous lecture …

- Deadlock detection and avoidance using the Banker's algorithm
- Sleepy Barbers problem

Muddiest Points

- Checked on Tophat

Sleepy Barbers Solution: Take 2

```
struct mysems {
   Semaphore RV1a(0), RV1b(0), RV2a(0), RV2b(0);
};
SharedBuffer buff; //producers-consumers problem
```

Worker Process

```
struct mysems sems = buff.consume();
up(sems.RV1a);
down(sems.RV1b);
//do work
down(sems.RV2a);
up(sems.RV2b);
//check-in for next customer
```

Customer Process

```
struct mysems sems = new struct mysems
buff.produce(sems);
down(sems.RV1a);
up(sems.RV1b);
//get work
up(sems.RV2a);
down(sems.RV2b);
//leave
```

Questions of the Day

- How to implement condition variables?
- Reflect more on all the solutions/problems that we have studied

User-level implementation of Condition Variables

A Lock with two waiting queues

```
struct Lock {
  Semaphore mutex(1);
   Semaphore next(0);
  int nextCount = 0;
}
Acquire(){
  mutex.down();
}
```

```
Release(){
   if(nextCount > 0){
      next.up();
      nextCount--;
  } else mutex.up();
}
```

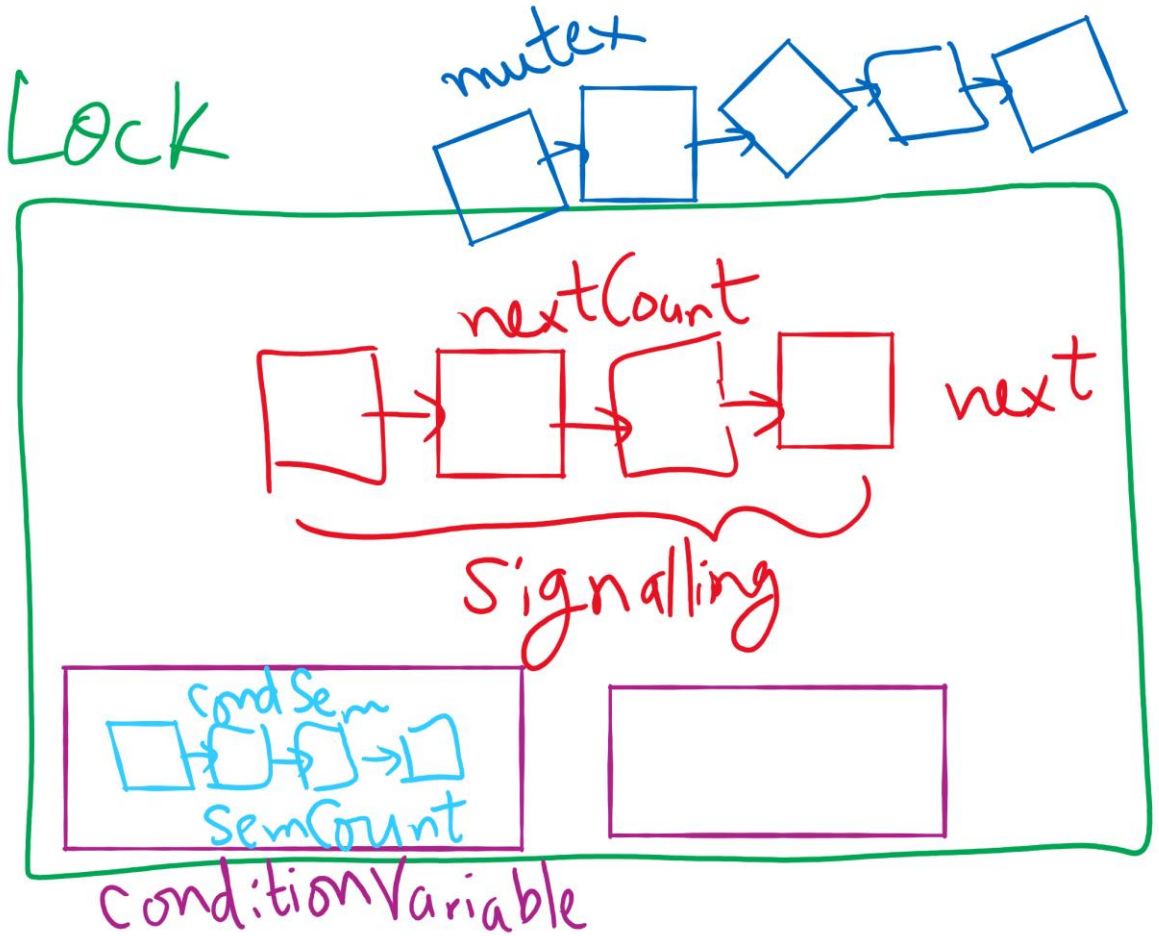
Condition Variable

```
struct ConditionVariable {
  Semaphore condSem(0);
   int semCount = 0;
  Lock *lk;
}
```

```
Wait(){
  if(lk->nextCount > 0)
     lk->next.up();
     lk->nextCount--;
  else {
     lk->mutex.up();
  }
  semCount++;
  condSem.down();
   semCount--;
}
```

```
Signal(){
  if(semCount > 0){
    condSem.up()
    lk->nextCount++
    lk->next.down();
    lk->nextCount—
  }
}
```

Lock and Condition Variable Implementation



Implementing locks with semaphores

- Use mutex to ensure exclusion within the lock bounds
- Use next to give lock to processes with a higher priority (why?)
- nextCount indicates whether there are any higher priority waiters

```
class Lock {
  Semaphore mutex(1);
  Semaphore next(0);
  int nextCount = 0;
};
```

```
Lock::Acquire()
{
  mutex.down();
}
```

```
Lock::Release()
{
  if (nextCount > 0)
    next.up();
  else
    mutex.up();
}
```

Implementing condition variables

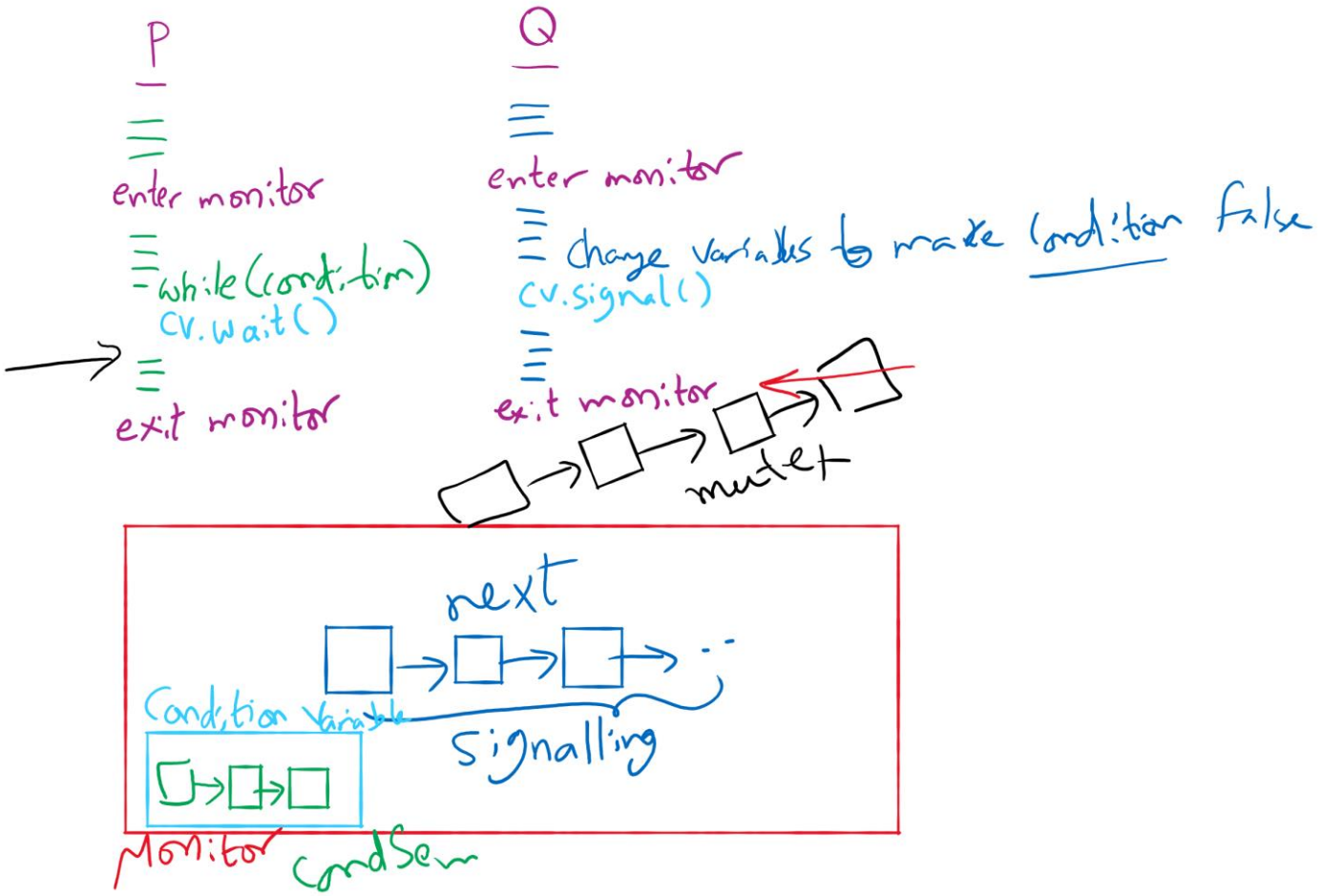
- Are these Hoare or Mesa semantics?
- Can there be multiple condition variables for a single Lock?

```
class Condition {
  Lock *lock;
  Semaphore condSem(0);
  int semCount = 0;
};
```

```
Condition::Wait ()
{
  semCount += 1;
  if (lock->nextCount > 0)
    lock->next.up();
  else
    lock->mutex.up();
  condSem.down ();
  semCount -= 1;
}
```

```
Condition::Signal ()
{
  if (semCount > 0) {
    lock->nextCount += 1;
    condSem.up ();
    lock->next.down ();
    lock->nextCount -= 1;
  }
}
```

Process Synchronization inside Monitors



Condition Variable-based Solutions

- Code Walkthrough at:

https://cs1550-2214.github.io/cs1550-code-handouts/ProcessSynchronization/Slides/

Reflections on semaphore usage

- Semaphores can be used as
  - Resource counters
  - Waiting spaces
  - For mutual exclusion

Reflections on Condition Variables

- Define a class and put all shared variables inside the class
- Include a mutex and a condition variable in the class
- For each public method of the class
  - Start by locking the mutex lock
  - If need to wait, use a while loop and wait on the condition variable
  - Before **broadcasting** on the condition variable, make sure to change the waiting condition